Design Patterns

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**

Structural

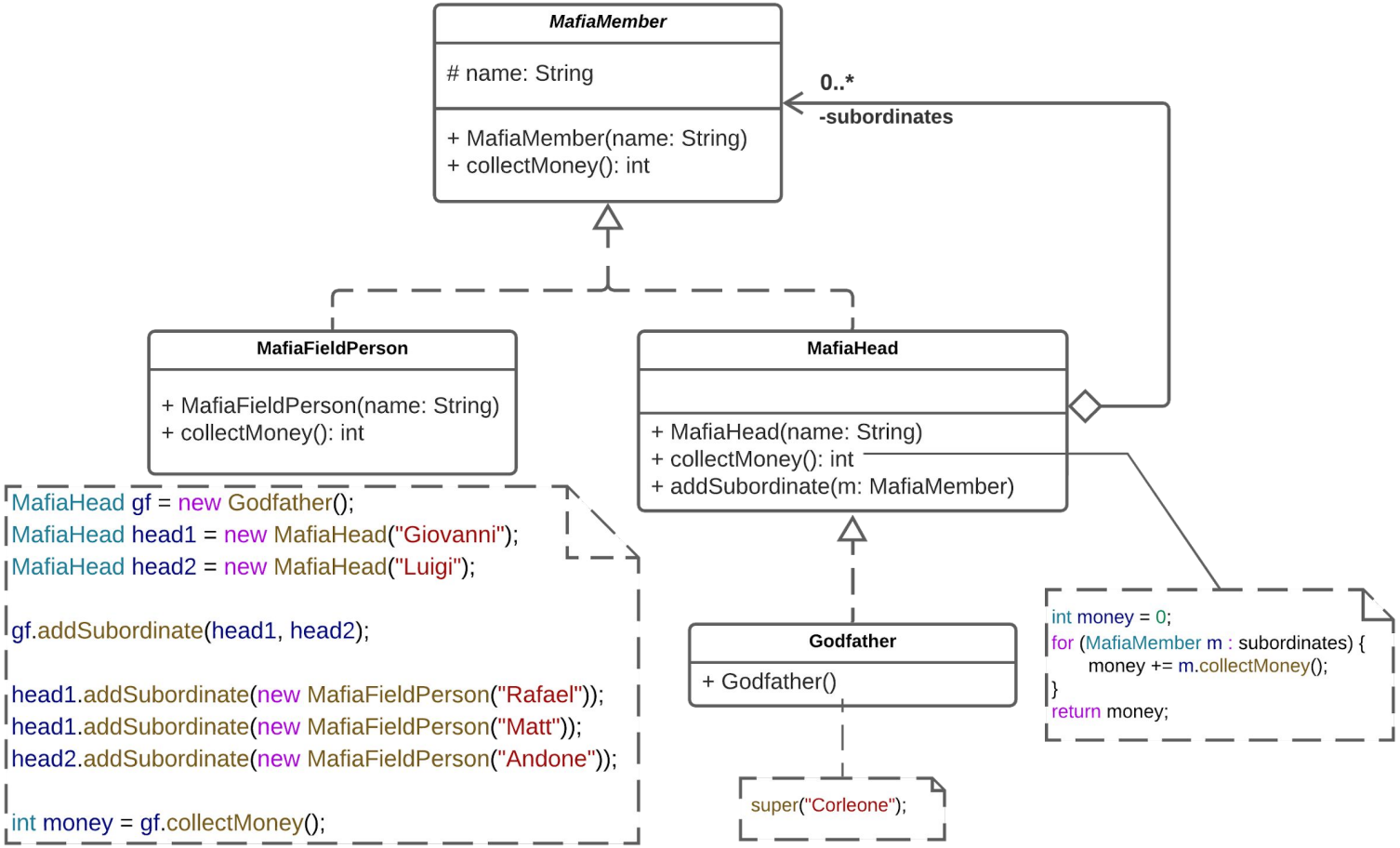
- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight

Behavioral

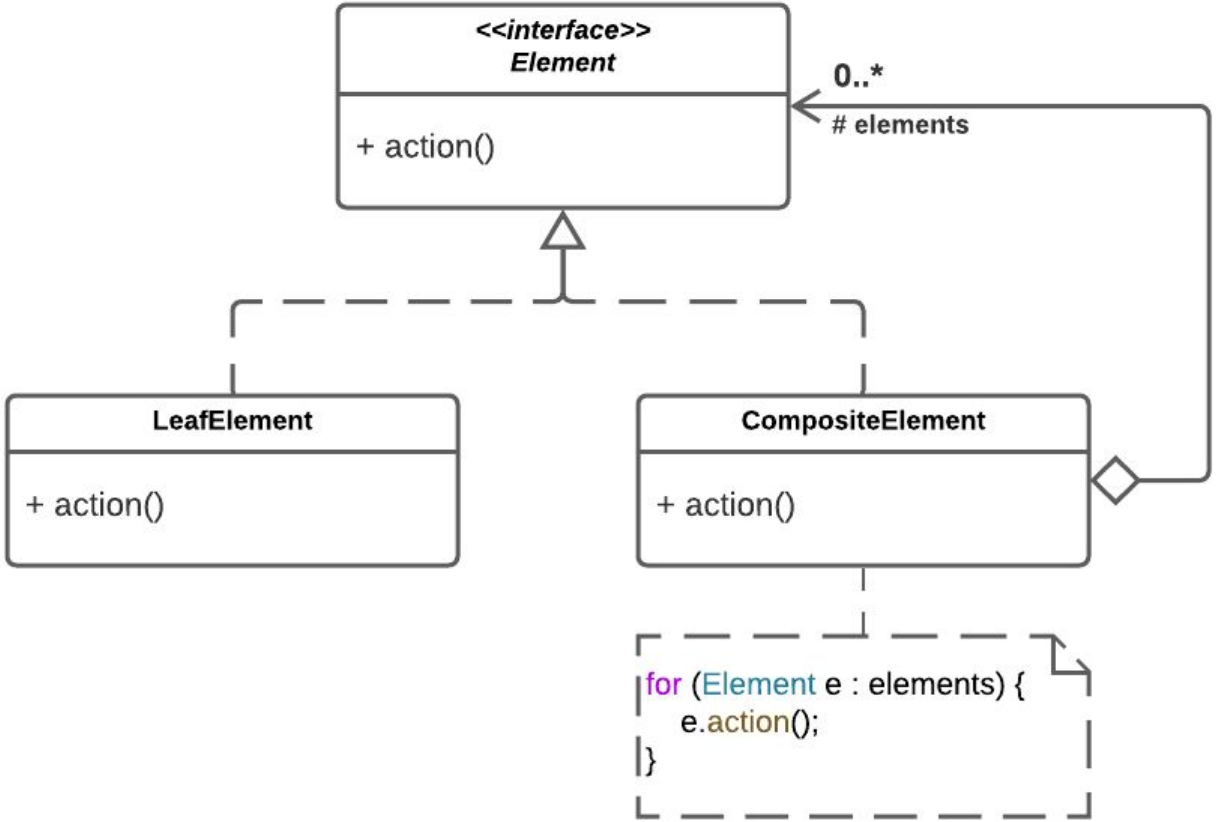
- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State

The Godfather Problem

Godfather implementation

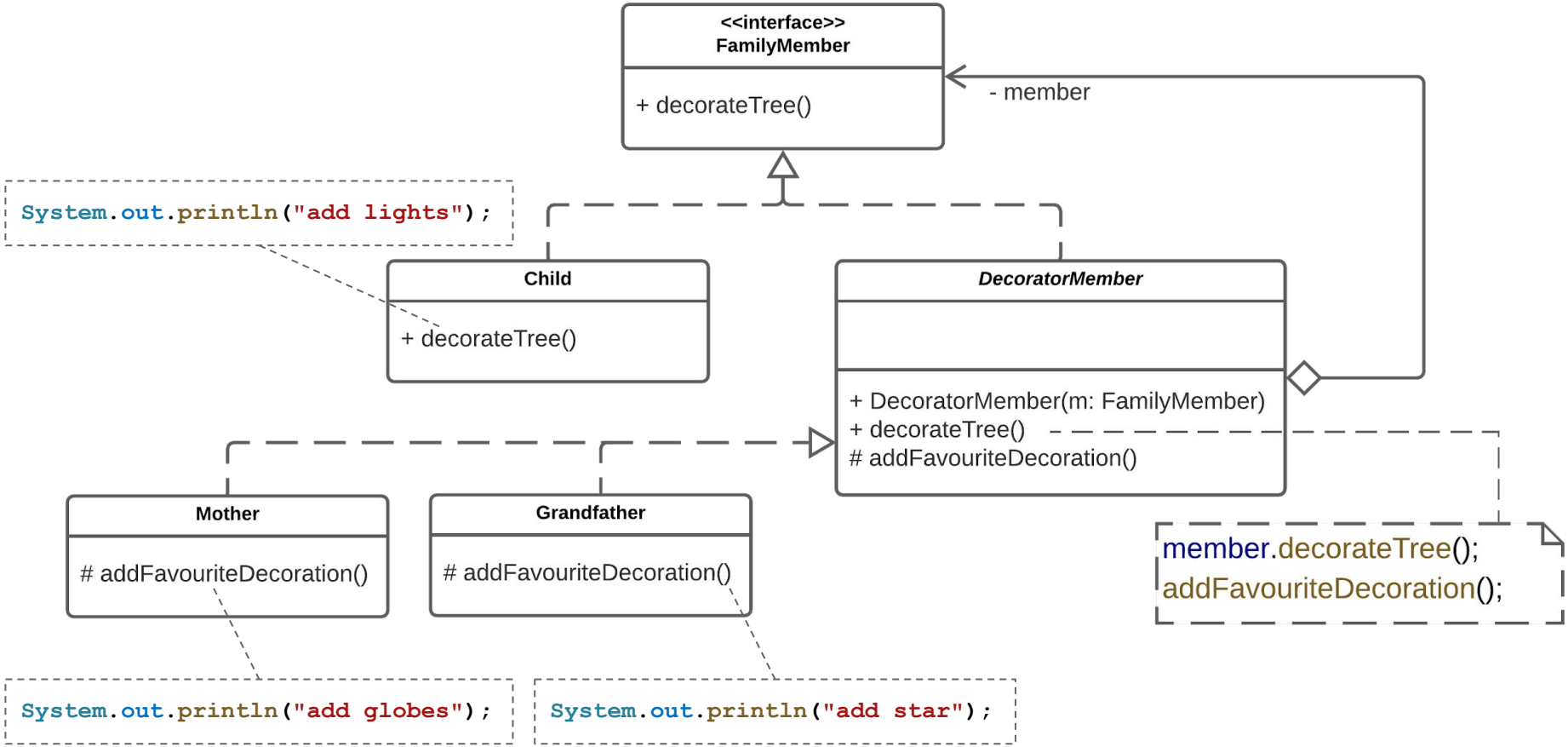


The Composite Pattern



The Family Christmas Tree Problem

The Family Christmas Tree Implementation

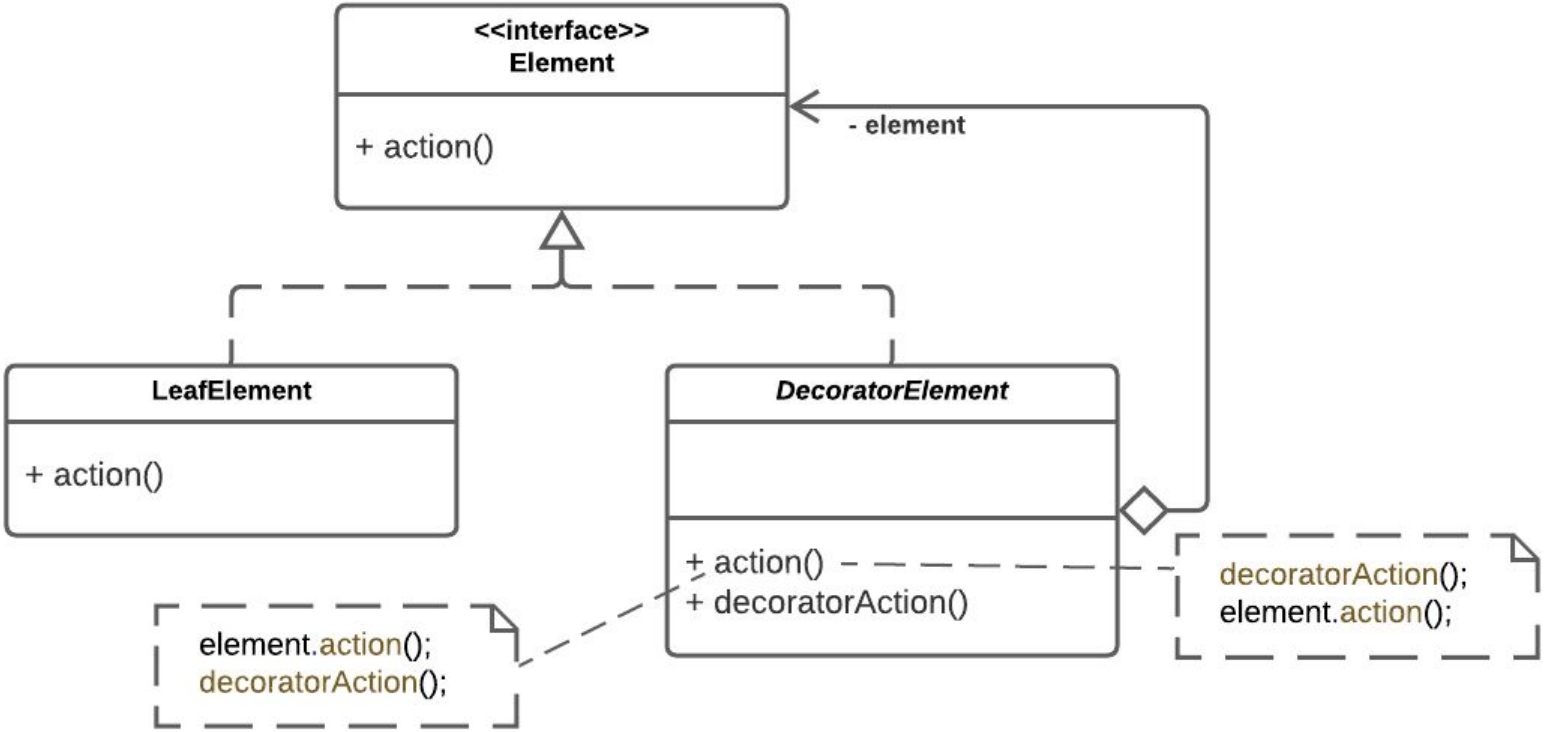


The Family Christmas Tree Implementation

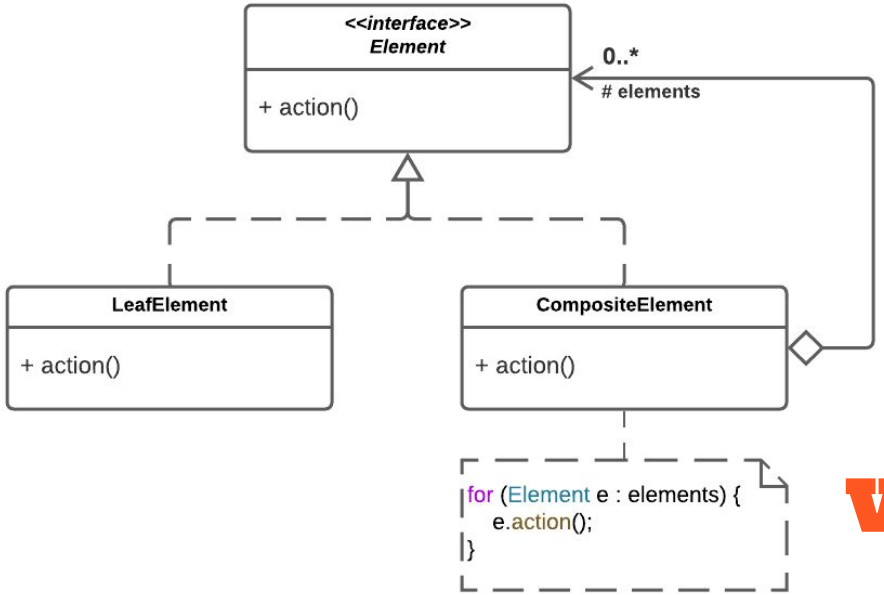
```java
interface FamilyMember {
    void decorateTree();
}
abstract class DecoratorElement implements FamilyMember
{
    private final FamilyMember member;

    public DecoratorElement(FamilyMember m) {
        this.member = m;
    }

    @Override
    public void decorateTree() {
        addFavouriteDecoration();
        member.decorateTree();
    }

    protected abstract void addFavouriteDecoration();
}
class Child implements FamilyMember {
    @Override
    public void decorateTree() {
        System.out.println("add lights");
    }
}
```

```java
class Mother extends DecoratorElement {
    public Mother(FamilyMember m) {
        super(m);
    }

    protected void addFavouriteDecoration() {
        System.out.println("add globes");
    }
}
class Grandfather extends DecoratorElement {
    public Grandfather(FamilyMember m) {
        super(m);
    }

    protected void addFavouriteDecoration() {
        System.out.println("add star");
    }
}
class Main {
    public static void main(String[] args) {
        new Grandfather(
            new Mother(
                new Child()
            )
        ).decorateTree();
    }
}
```

The Decorator Pattern



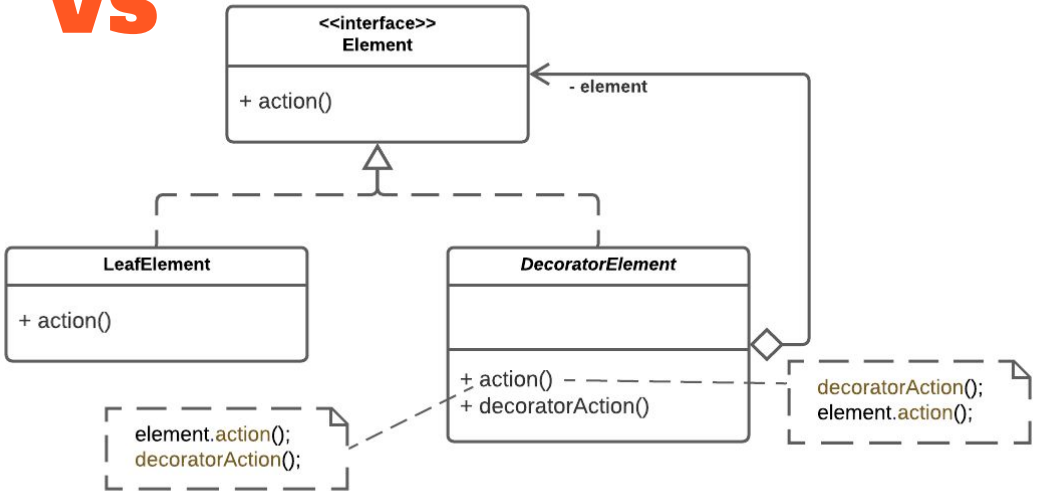
Structural Patterns Wrap up



Composite

VS

Decorator



Summary

- Composite is used to represent hierarchical structures
- Composite objects just delegate their responsibilities to their children and maybe aggregate the results
- Decorator is used to be able to dynamically add functionality to an object.
- A Decorator object lets his children do the job and then adds its own functionality

Creational

- Factory Method
- Abstract Factory
- Singleton
- Prototype (cloning)
- Builder

Structural

- Composite
- Decorator
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight

Behavioral

- Chain of responsibility
- Command
- Observer
- Template Method
- Strategy Pattern
- Visitor (+ Acyclic Visitor)
- Interpreter
- Iterator
- Mediator
- Memento
- State

Resources

- https://sourcemaking.com/design_patterns
- https://refactoring.guru/design-patterns
- ***Design Patterns: Elements of Reusable Object-Oriented Software***, written by *Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides*